# MLBD assignment 4: Clustering and PageRank

Github link : [https://github.com/kshreyash3/M25DE1016_CSL7110_Assignment4](https://github.com/kshreyash3/M25DE1016_CSL7110_Assignment4)

## Part 1: Clustering

```
Loading dataset...
Loaded 4601 points.

--- 1. kcenter (k=10) ---
kcenter execution time: 0.6869 seconds

--- 2. kmeansPP (k=10) ---
Returned value (kmeansObj for kmeansPP): 28156.6201

--- 3. Coreset Evaluation (k1=50, k=10) ---
Returned value (kmeansObj with coreset X): 341206.3424

Experiment Complete.
```

- **1. kcenter Time (0.6869 seconds):** Measures execution speed. It shows the Farthest-First algorithm quickly scanned all 4,601 points to find 10 highly dispersed centers.

- **2. kmeansPP Objective (28,156.62):** This is your baseline "error score" (lower is better). It represents the average squared distance from data points to their nearest center when kmeans++ evaluates the entire dataset.

- **3. Coreset Evaluation (341,206.34):** This error is roughly 12 times worse than the baseline. The kcenter algorithm intentionally selects extreme outliers to maximize distance. By limiting the coreset ($X$) to only 50 points, kmeans++ is forced to choose its final centers from these outliers. Because these centers sit on the extreme edges, they are far away from the majority of the normal data.

As k1 increases, kcenter runs out of extreme outliers and is forced to pick points in the denser, normal areas of the dataset. This creates a more representative coreset and drives the final error score down closer to the baseline.

## Part 2: Web-Search

```
--- Running Actions from actions.txt ---
No webpage contains word delhi
stack_datastructure_wiki
stack_datastructure_wiki
Webpage stack_datastructure_wiki does not contain word magazines
No webpage contains word allain
stack_cprogramming
stack_cprogramming
stack_cprogramming
stack_oracle
stack_cprogramming, stack_datastructure_wiki, stackoverflow
stackmagazine
```

### Web-Search - Understanding & Assumptions

- **Core Architecture:** Implemented an Inverted Index using Python dictionaries (acting as a Hash Table) to achieve $O(1)$ time complexity for word lookups. Python Sets were used to ensure webpages were not listed multiple times for a single search query.
- **Text Processing:** All text was converted to lowercase, and specific punctuation marks were replaced with spaces (rather than deleted) to prevent adjacent words from merging.
- **Connector Words:** Stop words (connectors) were excluded from the index to save space, but they were explicitly counted during processing to maintain accurate 1-based indexing for the valid words that followed them.
- **Normalization Assumption:** I assumed the provided singular-plural mapping (e.g., stacks -> stack) was completely exhaustive. No external NLP libraries were used for word stemming.

## Part 3: PageRank on Spark

```
--- Running PageRank on small.txt ---
Total nodes (n): 100
Running 40 iterations with beta=0.8...
Completed iteration 5...
Completed iteration 10...
Completed iteration 15...
Completed iteration 20...
Completed iteration 25...
Completed iteration 30...
Completed iteration 35...
Completed iteration 40...

Top 5 Nodes (Highest Scores):
Node: 53, Score: 0.035731
Node: 14, Score: 0.034171
Node: 40, Score: 0.033630
Node: 1, Score: 0.030006
Node: 27, Score: 0.029720

Bottom 5 Nodes (Lowest Scores):
Node: 85, Score: 0.003410
Node: 59, Score: 0.003670
Node: 81, Score: 0.003695
Node: 37, Score: 0.003808
Node: 89, Score: 0.003922
```

```python
# --- Execution ---
run_pagerank("small.txt", iterations=40, beta=0.8)
```

```
--- Running PageRank on small.txt ---
Total nodes (n): 100
Running 40 iterations with beta=0.8...
Completed iteration 5...
Completed iteration 10...
Completed iteration 15...
Completed iteration 20...
Completed iteration 25...
Completed iteration 30...
Completed iteration 35...
Completed iteration 40...

Top 5 Nodes (Highest Scores):
Node: 53, Score: 0.035731
Node: 14, Score: 0.034171
Node: 40, Score: 0.033630
Node: 1, Score: 0.030006
Node: 27, Score: 0.029720

Bottom 5 Nodes (Lowest Scores):
Node: 85, Score: 0.003410
Node: 59, Score: 0.003670
Node: 81, Score: 0.003695
Node: 37, Score: 0.003808
Node: 89, Score: 0.003922
```

**Concept & Assumptions:** Implemented iterative PageRank (40 iterations, $\beta = 0.8$) using PySpark. I assumed multiple directed edges between the same nodes should be treated as a single edge (handled via .distinct()). To prevent "dead-end" nodes from trapping the score, every node received a baseline teleportation probability of (1 - 0.8) / n during each iteration.

**Validation:** Successfully verified the logic on small.txt, matching the expected top score of ~0.036.

**Top Nodes** (whole.txt)**:** Node 53 and Node 454 achieved the highest scores, indicating they act as central "hubs" receiving the most incoming edge weight from other important nodes.

**Bottom Nodes** (whole.txt)**:** The lowest-scoring nodes (93, 540, 162, 714, 45) all tied at exactly **0.000200**. This perfectly matches the base teleportation formula (1 - 0.8) / 1000 = 0.0002. This mathematically proves these nodes have an in-degree of exactly zero—they receive no incoming links and their score is purely from random teleportation.